

第4章 贪心算法

学习要点

- 理解贪心算法的概念。
- 掌握贪心算法的基本要素
 - (1) 最优子结构性质
 - (2) 贪心选择性质
- 理解贪心算法与动态规划算法的差异
- 理解贪心算法的一般理论
- 通过应用范例学习贪心设计策略
 - (1) 活动安排问题；
 - (2) 最优装载问题；
 - (3) 哈夫曼编码；
 - (4) 单源最短路径；
 - (5) 最小生成树；
 - (6) 多机调度问题。

简单例子

1. 硬币4种：2角5分、1角、5分、1分。求解最小的硬币个数，使硬币总和为6角3分。

2角5分： 2个

1角： 1个

1分： 3个

2. 硬币3种：1分、5分、1角1分。求解最小硬币个数，使硬币综合为1角5分

3个5分 好于 1角1分+4个1分

贪心算法总是作出在当前看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的**局部最优**选择。当然，希望贪心算法得到的最终结果也是**整体最优**的。

虽然贪心算法不能对所有问题都得到整体最优解，但对许多问题它能产生整体最优解。如**单源最短路径**问题，**最小生成树**问题等。

在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是最优解的很好近似。

4.1 活动安排问题

活动安排问题就是要在所给的活动集合中选出最大的相容活动子集合，是可以用**贪心算法**有效求解的很好例子。该问题要求高效地安排一系列争用某一公共资源的活动。贪心算法提供了一个简单、漂亮的方法使得尽可能多的活动能兼容地使用公共资源。

4.1 活动安排问题

设有 n 个活动的集合 $E=\{1,2,\dots,n\}$ ，其中每个活动都要求使用同一资源，如演讲会场等，而在同一时间内只有一个活动能使用这一资源。每个活动 i 都有一个要求使用该资源的起始时间 s_i 和一个结束时间 f_i ，且 $s_i < f_i$ 。如果选择了活动 i ，则它在半开时间区间 $[s_i, f_i)$ 内占用资源。若区间 $[s_i, f_i)$ 与区间 $[s_j, f_j)$ 不相交，则称活动 i 与活动 j 是相容的。也就是说，当 $s_i \geq f_j$ 或 $s_j \geq f_i$ 时，活动 i 与活动 j 相容。

4.1 活动安排问题

下面给出解活动安排问题的贪心算法**GreedySelector** :

```
template<class Type>
void GreedySelector(int n, Type s[], Type f[], bool A[])
{
    A[1]=true;
    int j=1;
    for (int i=2;i<=n;i++) {
        if (s[i]>=f[j]) { A[i]=true; j=i; }
        else A[i]=false;
    }
}
```

各活动的起始时间和结束时间存储于数组s和f中且按结束时间的非减序排列

4.1 活动安排问题

由于输入的活动以其完成时间的**非减序**排列，所以算法 **greedySelector** 每次总是选择**具有最早完成时间**的相容活动加入集合A中。直观上，按这种方法选择相容活动为未安排活动留下尽可能多的时间。也就是说，该算法的贪心选择的意义是**使剩余的可安排时间段极大化**，以便安排尽可能多的相容活动。

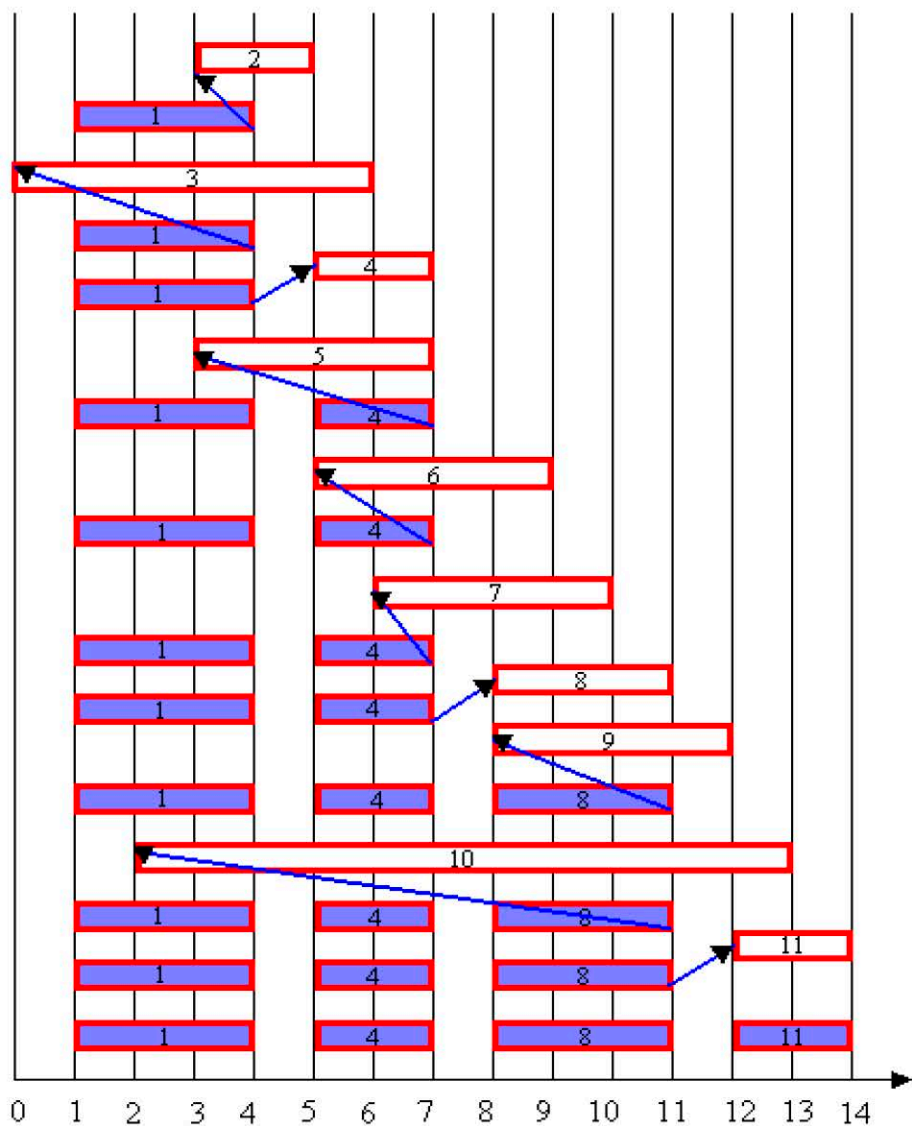
当输入的活动已按结束时间的非减序排列，算法只需 **$O(n)$** 的时间安排n个活动，使最多的活动能相容地使用公共资源。如果所给出的活动未按非减序排列，可以用 **$O(n\log n)$** 的时间重排。

4.1 活动安排问题

例：设待安排的11个活动的开始时间和结束时间，按结束时间的非减序排列如下：

i	1	2	3	4	5	6	7	8	9	10	11
s[i]	1	3	0	5	3	5	6	8	8	2	12
f[i]	4	5	6	7	8	9	10	11	12	13	14

4.1 活动安排问题



算法greedySelector 的计算过程如左图所示。图中每行相应于算法的一次迭代。阴影长条表示的活动是已选入集合A的活动，而空白长条表示的活动是当前正在检查相容性的活动。

4.1 活动安排问题

若被检查的活动 i 的开始时间 S_i 小于最近选择的活动的结束时间 f_j ，则不选择活动 i ，否则选择活动 i 加入集合 A 中。

贪心算法greedySelector却总能求得的整体最优解，即它最终所确定的相容活动集合 A 的规模最大。这个结论可以用数学归纳法证明。

4.2 贪心算法的基本要素

对于一个具体的问题，怎么知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢？这个问题很难给予肯定的回答。

但是，从许多可以用贪心算法求解的问题中看到这类问题一般具有2个重要的性质：

1) 贪心选择性质

2) 最优子结构性质

4.2 贪心算法的基本要素

1、贪心选择性质

所谓**贪心选择性质**是指所求问题的**整体最优解**可以通过一系列**局部最优**的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。

动态规划算法通常以**自底向上**的方式解各子问题，而贪心算法则通常以**自顶向下**的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。

对于一个具体问题，要确定它**是否具有贪心选择性质**，必须证明每一步所作的贪心选择最终导致问题的整体最优解。

4.2 贪心算法的基本要素

2、最优子结构性质

当一个问题的最优解包含其子问题的最优解时，称此问题具有**最优子结构性质**。问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。

4.2 贪心算法的基本要素

3、贪心算法与动态规划算法的差异

贪心算法和动态规划算法都要求问题具有最优子结构性质，这是2类算法的一个共同点。但是，

1) 对于具有**最优子结构**的问题应该选用贪心算法还是动态规划算法求解？

2) 是否能用动态规划算法求解的问题也能用贪心算法求解？

下面研究2个经典的**组合优化问题**，并以此说明贪心算法与动态规划算法的主要差别。

4.2 贪心算法的基本要素

■ 0-1背包问题：

给定 n 种物品和一个背包。物品 i 的重量是 W_i ，其价值为 V_i ，背包的容量为 C 。应如何选择装入背包的物品，使得装入背包中物品的总价值最大？

在选择装入背包的物品时，对每种物品 i 只有2种选择，即装入背包或不装入背包。不能将物品 i 装入背包多次，也不能只装入部分的物品 i 。

4.2 贪心算法的基本要素

■ 背包问题：

与0-1背包问题类似，所不同的是在选择物品 i 装入背包时，**可以选择物品 i 的一部分**，而不一定要全部装入背包， $1 \leq i \leq n$ 。

这2类问题都具有**最优子结构**性质，极为相似，但背包问题可以用贪心算法求解，而0-1背包问题却不能用贪心算法求解。

4.2 贪心算法的基本要素

用贪心算法解背包问题的基本步骤：

首先计算每种物品单位重量的价值 V_i/W_i ，然后，依贪心选择策略，将尽可能多的**单位重量价值最高**的物品装入背包。

若将这种物品全部装入背包后，背包内的物品总重量未超过 C ，则选择单位重量价值次高的物品并尽可能多地装入背包。

依此策略一直地进行下去，直到背包装满为止。

4.2 贪心算法的基本要素

```
void Knapsack(int n,float M,float v[],float w[],float x[])
```

```
{
```

```
    Sort(n,v,w);
```

```
    int i;
```

```
    for (i=1;i<=n;i++) x[i]=0;
```

```
    float c=M;
```

```
    for (i=1;i<=n;i++) {
```

```
        if (w[i]>c) break;
```

```
        x[i]=1;
```

```
        c-=w[i];
```

```
    }
```

```
    if (i<=n) x[i]=c/w[i];
```

```
}
```

算法knapsack的主要计算时间在于将各种物品依其单位重量的价值从大到小排序。因此，算法的计算时间上界为 $O(n \log n)$ 。为了证明算法的正确性，还必须证明背包问题具有贪心选择性质。

4.2 贪心算法的基本要素

对于**0-1背包问题**，贪心选择之所以不能得到最优解是因为在这种情况下，它**无法保证最终能将背包装满**，部分闲置的背包空间使每公斤背包空间的价值降低了。事实上，在考虑0-1背包问题时，应比较选择该物品和不选择该物品所导致的最终方案，然后再作出最好选择。由此就导出许多互相重叠的子问题。这正是该问题可用**动态规划算法**求解的另一重要特征。

实际上也是如此，动态规划算法的确可以有效地解0-1背包问题。

4.3 最优装载

有一批集装箱要装上一艘载重量为 c 的轮船。其中集装箱 i 的重量为 W_i 。最优装载问题要求确定在装载体积不受限制的情况下，将尽可能多的集装箱装上轮船。

1、算法描述

最优装载问题可用贪心算法求解。采用**重量最轻者先装的贪心选择策略**，可产生最优装载问题的最优解。

4.3 最优装载

```
template<class Type>
void Loading(int x[], Type w[], Type c, int n)
{
    int *t = new int [n+1];
    Sort(w, t, n);
    for (int i = 1; i <= n; i++) x[i] = 0;
    for (int i = 1; i <= n && w[t[i]] <= c; i++) {
        x[t[i]] = 1;
        c -= w[t[i]];
    }
}
```

4.3 最优装载

2、贪心选择性质

可以证明最优装载问题具有贪心选择性质。

3、最优子结构性质

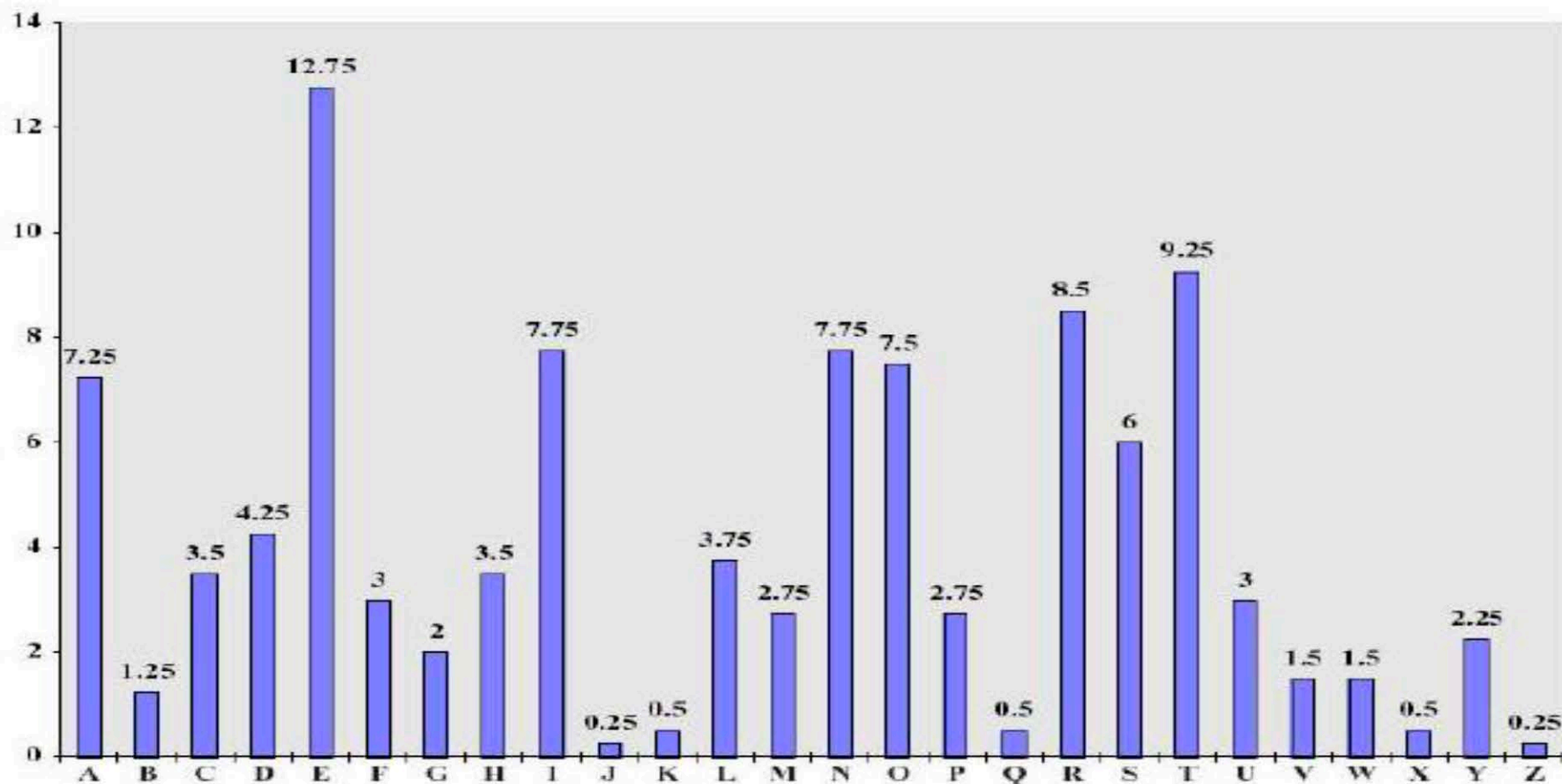
最优装载问题具有最优子结构性质。

由最优装载问题的贪心选择性质和最优子结构性质，容易证明算法**loading**的正确性。

算法**loading**的主要计算量在于将集装箱依其重量从小到大排序，故算法所需的计算时间为 **$O(n\log n)$** 。

4.4 哈夫曼编码

数据文件压缩的思考



4.4 哈夫曼编码

	a	b	c	d	e	f
频率（千次）	45	13	12	16	9	5
定长码	000	001	010	011	100	101
变长码	0	101	100	111	1101	1100

◆ 定长码：300000 位

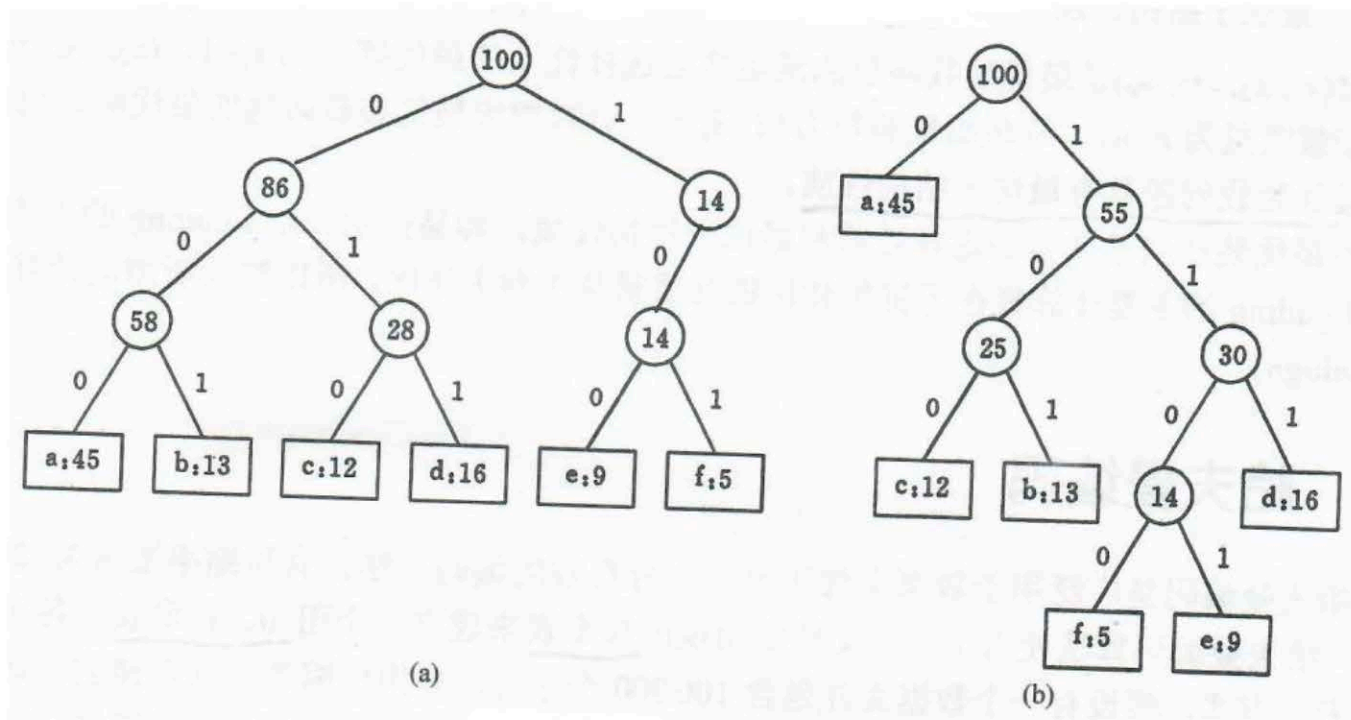
◆ 变长码：224000 位

001 011 101 定长码： bdf

0、0、101、1101 变长码： aabe

4.4 哈夫曼编码

	a	b	c	d	e	f
频率 (千次)	45	13	12	16	9	5
定长码	000	001	010	011	100	101
变长码	0	101	100	111	1101	1100



4.4 哈夫曼编码

哈夫曼编码是广泛地用于数据文件压缩的十分有效的编码方法。其压缩率通常在20% ~ 90%之间。哈夫曼编码算法用字符在文件中出现的频率表来建立一个用0、1串表示各字符的最优表示方式。

给**出现频率高的字符较短的编码**，**出现频率较低的字符以较长的编码**，可以大大缩短总码长。

4.4 哈夫曼编码

◆ 前缀码

对每一个字符规定一个0,1串作为其代码，并要求任一字符的代码都不是其它字符代码的前缀。这种编码称为**前缀码**。

◆ 最优前缀码

使平均码长达到最小的前缀码编码方案称为给定编码字符集C的最优前缀码。

平均码长定义为：
$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

表示**最优前缀码**的二叉树总是一棵**完全二叉树**，即树中任一结点都有2个儿子结点。

4.4 哈夫曼编码

- ◆ 哈夫曼编码

哈夫曼提出构造最优前缀码的贪心算法，由此产生的编码方案称为**哈夫曼编码**。

哈夫曼算法以自底向上的方式构造表示最优前缀码的二叉树T。

算法以 $|C|$ 个叶结点开始，执行 $|C| - 1$ 次的“合并”运算后产生最终所要求的树T。

4.4 哈夫曼编码

HuffmanTree中，编码字符集中每一字符 c 的频率是 $f(c)$ 。
以 f 为键值的优先队列 Q 用在**贪心选择**时有效地确定算法当前要合并的2棵具有最小频率的树。一旦2棵具有最小频率的树合并后，产生一棵新的树，其频率为合并的2棵树的频率之和，并将新树插入优先队列 Q 。经过 $n - 1$ 次的合并后，优先队列中只剩下一棵树，即所要求的树 T 。

4.4 哈夫曼编码

```
MinHeap<Huffman<type>> Q(1); //建立优先队列
```

```
Q.Initialize(w, n, n);
```

```
//反复合并最小频率树
```

```
Huffman<Type> x, y;
```

```
for (int i=1; i<n; i++) {
```

```
    Q.DeleteMin(x);
```

```
    Q.DeleteMin(y);
```

```
    z.MakeTree(0, x.tree, y.tree);
```

```
    x.weight += y.weight;
```

```
    x.tree = z;
```

```
    Q.insert(x);
```

```
}
```

4.4 哈夫曼编码

算法huffmanTree用最小堆实现优先队列Q。初始化优先队列需要 $O(n)$ 计算时间，由于最小堆的removeMin和put运算均需 $O(\log n)$ 时间， $n - 1$ 次的合并总共需要 $O(n \log n)$ 计算时间。因此，关于 n 个字符的哈夫曼算法的**计算时间为 $O(n \log n)$** 。

4.4 哈夫曼编码

3、哈夫曼算法的正确性

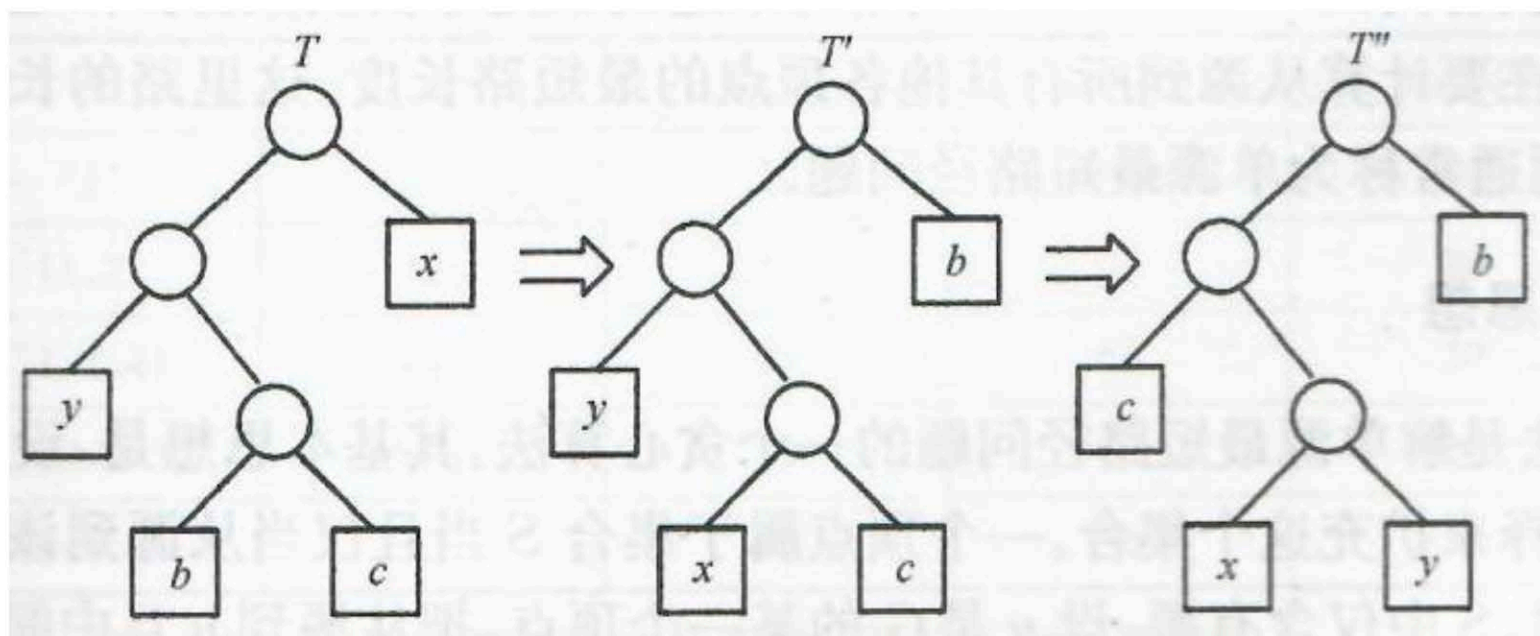
要证明哈夫曼算法的正确性，只要证明最优前缀码问题具有：

(1)贪心选择性质

(2)最优子结构性质

贪心选择性质

x和y是出现最小频率的两个字符

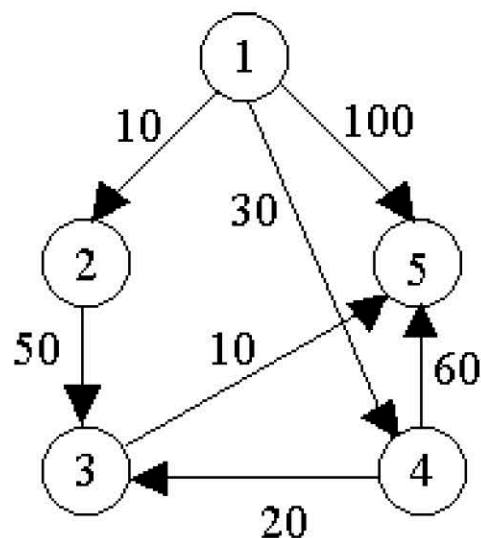


$$\begin{aligned} B(T) - B(T') &= \sum_{c \in C} f(c) d_T(c) - \sum_{c \in C} f(c) d_{T'}(c) \\ &= f(x) d_T(x) + f(b) d_T(b) - f(x) d_{T'}(x) - f(b) d_{T'}(b) \\ &= f(x) d_T(x) + f(b) d_T(b) - f(x) d_T(b) - f(b) d_T(x) \\ &= (f(b) - f(x))(d_T(b) - d_T(x)) \geq 0 \end{aligned}$$

4.5 单源最短路径

单源最短路径问题：给定带权有向图 $G=(V,E)$ ，其中每条边的权是非负实数。另外，还给定 V 中的一个顶点，称为源。现在要计算从**源到所有其它各顶点的最短路长度**，这里路的长度是指路上各边权之和。

Dijkstra算法是解**单源最短路径问题的贪心算法**。



4.5 单源最短路径

设置顶点集合 S 并不断地作**贪心选择**来扩充这个集合。一个顶点属于集合 S 当且仅当从源到该顶点的最短路径长度已知。

初始时， S 中仅含有源。设 u 是 G 的某一个顶点，把从源到 u 且中间只经过 S 中顶点的路称为从源到 u 的特殊路径，并用数组 $dist$ 记录当前每个顶点所对应的最短特殊路径长度。

Dijkstra算法每次从 $V-S$ 中**取出具有最短特殊路长度的顶点 u ，将 u 添加到 S 中**，同时对数组 $dist$ 作必要的修改。一旦 S 包含了所有 V 中顶点， $dist$ 就记录了从源到所有其它顶点之间的最短路径长度。

4.5 单源最短路径

```
dist[v]=0, s[v] =true;    //初始化
```

```
for (int i=1; i<n; i++) {    //遍历n-1次, 逐渐扩充S
```

```
    int temp = maxint, u = v;
```

```
    for (int j=1; j<=n; j++)    // 找与S距离最近的节点u
```

```
        if ( (!s[j]) && (dist[j] < temp) ) { u = j; temp = dist[j]; }
```

```
    s[u] =true;
```

```
    for(int j=1; j<=n; j++) {    // 根据新加入S的节点 u, 更新与不在S中的节点 j 的距离
```

```
        if( (!s[j]) && (c[u][j] < maxint)) {
```

```
            Type newdist = dist[u] + c[u][j];
```

```
            if ( newdist < dist[j] )
```

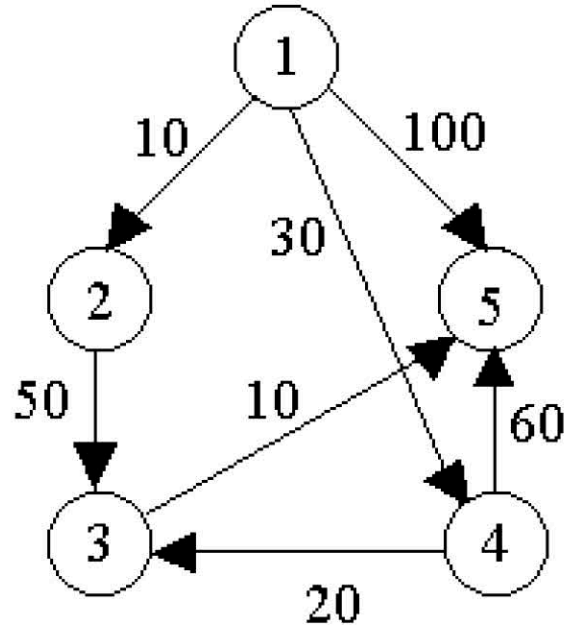
```
                { dist[j] = newdist; prev[j] = u; } //prev[j]为节点 j 在路径上的前一个节点
```

```
        }
```

```
    }
```

4.5 单源最短路径

Dijkstra算法的迭代过程：



迭代	S	u	dist[2]	dist[3]	dist[4]	dist[5]
初始	{1}	-	10	maxint	30	100
1	{1,2}	2	10	60	30	100
2	{1,2,4}	4	10	50	30	90
3	{1,2,4,3}	3	10	50	30	60
4	{1,2,4,3,5}	5	10	50	30	60

4.5 单源最短路径

2、算法的正确性和计算复杂性

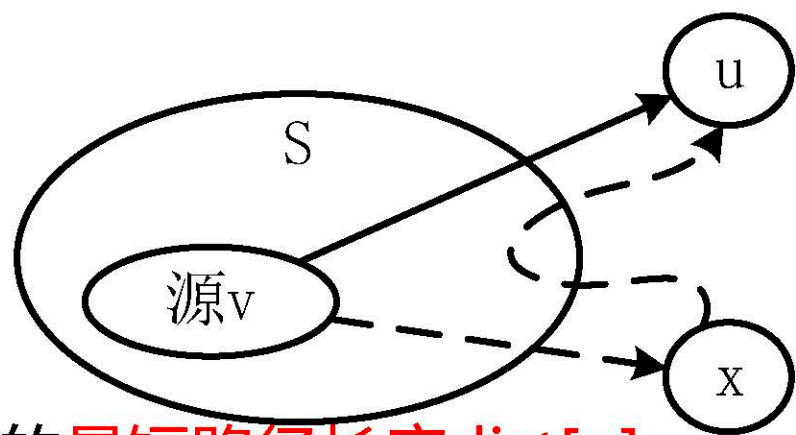
1)贪心选择性质

2)最优子结构性质

3)计算复杂性

对于具有 n 个顶点和 e 条边的带权有向图，如果用带权邻接矩阵表示这个图，那么Dijkstra算法的主循环体需要 $O(n)$ 时间。这个循环需要执行 $n-1$ 次，所以完成循环需要 $O(n^2)$ 时间。算法的其余部分所需要时间不超过 $O(n^2)$ 。

贪心选择性质



设从源到 u （不在 S 的所有节点）的**最短路径长度** $\text{dist}[u]$ 。

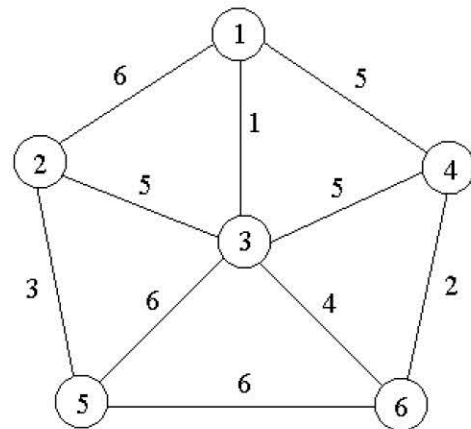
如果存在一条从源 v 到 u 且长度比 $\text{dist}[u]$ 更短的路，设这条路初次走出 S 之外到达的顶点为 $x \in V - S$ ，然后徘徊于 S 内外若干次，最后离开 S 到达 u 。

在这条路上，分别记 $d(v, x)$, $d(x, u)$, $d(v, u)$ 为顶点 v 到顶点 x ，顶点 x 到顶点 u ，顶点 v 到顶点 u 的路长。

$$\text{dist}[x] \leq d(v, x), \quad d(v, x) + d(x, u) = d(v, u) < \text{dist}[u].$$

由于边权非负，即 $d(x, u) \geq 0$ ，从而**推得** $\text{dist}[x] < \text{dist}[u]$ ，与假设**矛盾**。

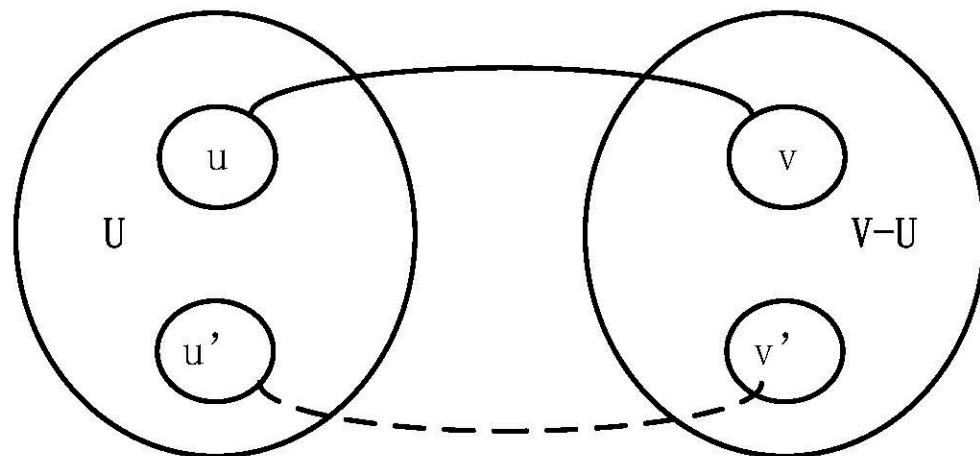
4.6 最小生成树



设 $G = (V, E)$ 是无向连通带权图，即一个网络。 E 中每条边 (v, w) 的权为 $c[v][w]$ 。如果 G 的子图 G' 是一棵包含 G 的所有顶点的树，则称 G' 为 G 的生成树。**生成树上各边权的总和称为该生成树的耗费**。在 G 的所有生成树中，耗费最小的生成树称为 G 的**最小生成树**。

网络的最小生成树在实际中有广泛应用。例如，在设计通信网络时，用图的顶点表示城市，用边 (v, w) 的权 $c[v][w]$ 表示建立城市 v 和城市 w 之间的通信线路所需的费用，则最小生成树就给出了建立通信网络的最经济的方案。

4.6 最小生成树



1、最小生成树性质

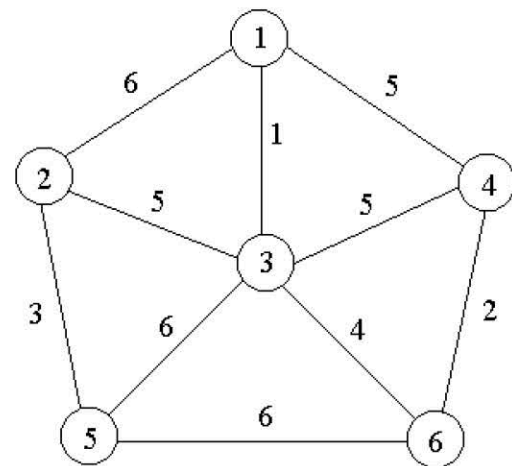
用贪心算法设计策略可以设计出构造最小生成树的有效算法。本节介绍的构造最小生成树的**Prim算法**和**Kruskal算法**都可以看作是应用贪心算法设计策略的例子。尽管这2个算法做贪心选择的方式不同，它们都利用了下面的**最小生成树性质**MST：

设 $G=(V,E)$ 是连通带权图， U 是 V 的真子集。如果 $(u,v) \in E$ ，且 $u \in U$ ， $v \in V-U$ ，且在所有这样的边中， (u,v) 的权 $c[u][v]$ 最小，那么一定存在 G 的一棵最小生成树，它以 (u,v) 为其中一条边。

4.6 最小生成树

2、Prim算法

设 $G=(V,E)$ 是连通带权图, $V=\{1,2,\dots,n\}$



构造 G 的最小生成树的Prim算法的基本思想是：首先置 $S=\{1\}$ ，然后，只要 S 是 V 的真子集，就作如下的**贪心选择**：

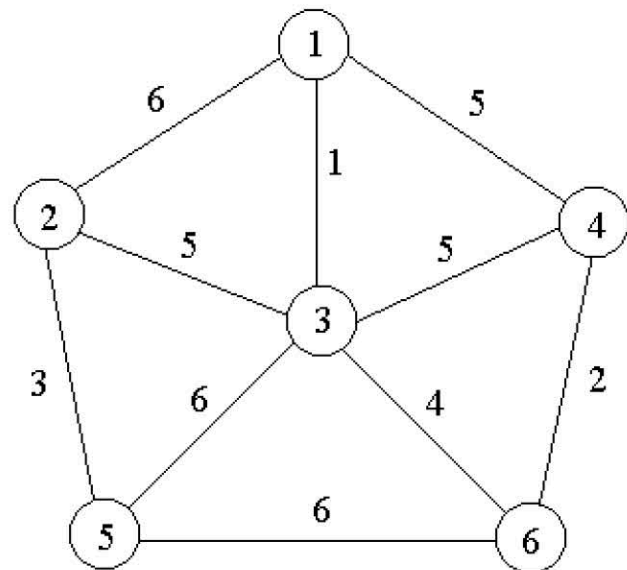
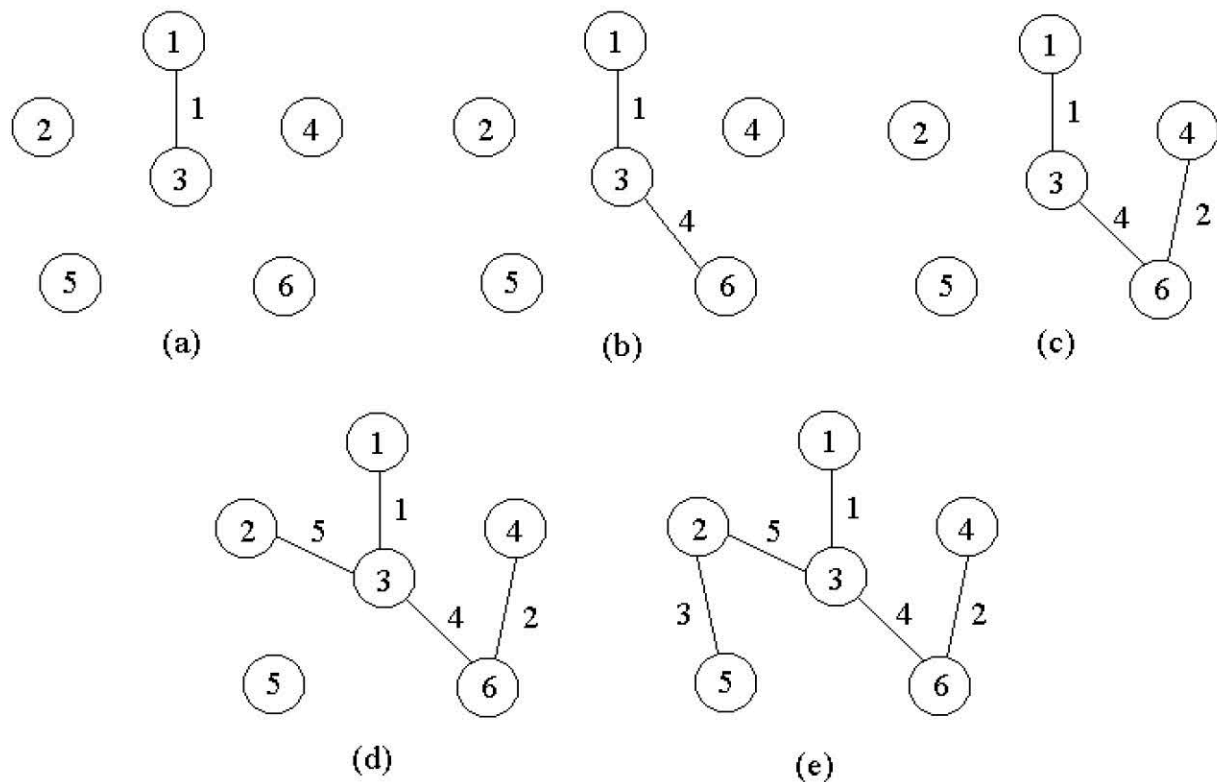
①选取满足条件 $i \in S, j \in V-S$ ，且 **$c[i][j]$ 最小的边，将顶点 j 添加到 S 中**。更新 S 中节点与其它节点距离。

②这个过程一直进行到 $S=V$ 时为止。

在这个过程中选取到的所有边恰好构成 G 的一棵最小生成树。

4.6 最小生成树

Prim算法**选取边**的过程



4.6 最小生成树

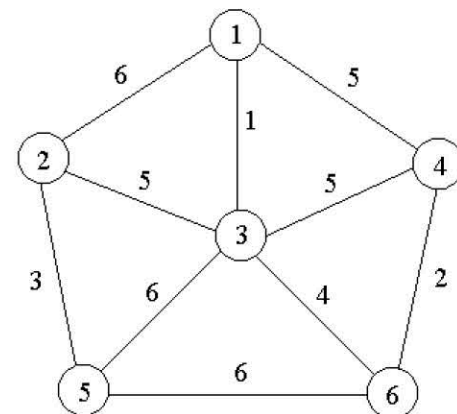
在Prim算法中，应当考虑**如何有效地找出满足条件 $i \in S, j \in V-S$ ，且权 $c[i][j]$ 最小的边 (i,j)** 。实现这个目的的较简单的办法是设置2个数组closest和lowcost。

在Prim算法执行过程中，先找出V-S中使lowcost值最小的顶点j，然后根据数组closest选取边(j, closest[j])，最后将j添加到S中，并对closest和lowcost作必要的修改。

用这个办法实现的Prim算法所需的**计算时间**为 $O(n^2)$

4.6 最小生成树

```
for (int i = 1; i < n; i++) { //循环n-1次
    Type min = inf, j = 1;
    for(int k=2; k <= n; k++) {
        if((lowcost[k] < min) && (!s[k])) {
            min = lowcost[k];
            j = k; //选择加入S的节点
        }
    }
    cout << j << ' ' << closet[j] << endl;
    s[j] = true;
    for (int k = 2; k <= n; k++) { // 更新S中的节点与其它节点距离
        if((c[j][k] < lowcost[k]) && (!s[k])) {
            lowcost[k] = c[j][k];
            closet[k] = j;
        }
    }
}
```



4.6 最小生成树

3、Kruskal算法

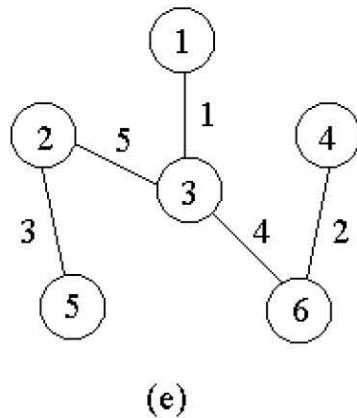
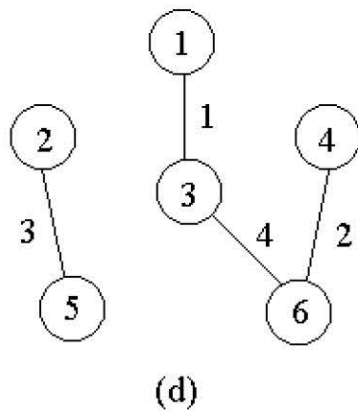
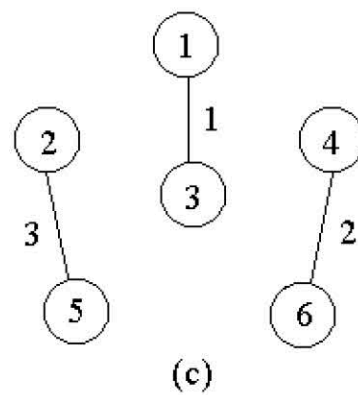
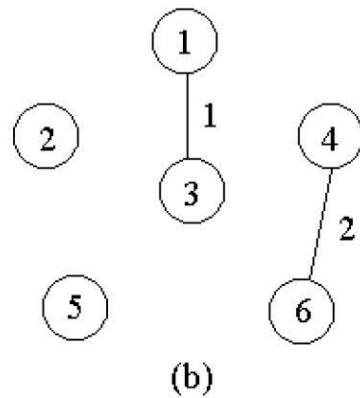
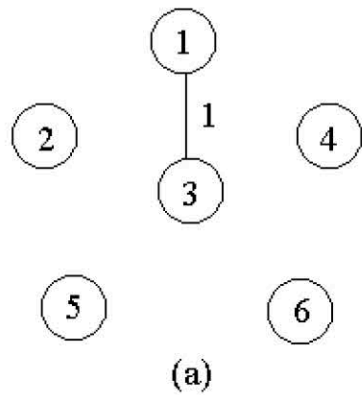
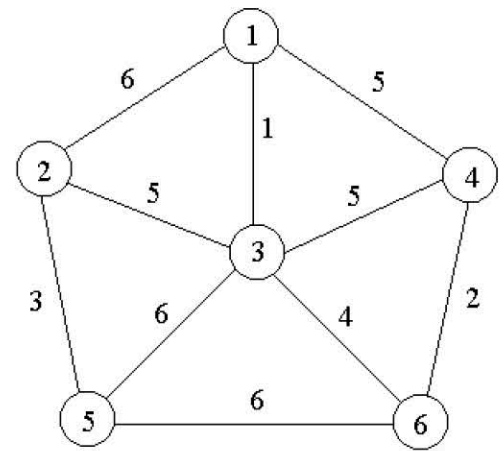
Kruskal算法构造G的最小生成树的基本思想：首先将G的n个顶点看成n个孤立的连通分支。将**所有的边按权从小到大排序**。然后从第一条边开始，**依边权递增的顺序查看每一条边**，并按下述方法连接2个不同的连通分支：

①当查看到第k条边(v,w)时，如果端点v和w分别是当前2个不同的连通分支T1和T2中的顶点时，**就用边(v,w)将T1和T2连接成一个连通分支，然后继续查看第k+1条边**；

②如果端点v和w在当前的同一个连通分支中，就直接再查看第k+1条边。这个过程一直进行到只剩下一个连通分支时为止。

4.6 最小生成树

Kruskal算法顺序得到的最小生成树



4.6 最小生成树-Kruska算法

```
while( e && k < n-1){  
    MinHeap<EdgeNode<Type>> H(1);  
    H.DeleteMin(x);  
    e--;  
    int a = U.Find(x.u);  
    int b = U.Find(x.v);  
    if(a != b) {  
        t[k++] = x;  
        U.Union(a,b);  
    }  
}
```

4.6 最小生成树

关于集合的一些基本运算可用于实现Kruskal算法。

按权的递增顺序查看等价于对**优先队列**执行**removeMin**运算。可以用堆实现这个优先队列。

对一个由连通分支组成的集合不断进行修改，需要用到抽象数据类型并查集**UnionFind**所支持的基本运算。

当图的边数为 e 时，Kruskal算法所需的计算时间是 $O(e \log e)$
当 $e = \Omega(n^2)$ 时，Kruskal算法比Prim算法差，但当 $e = o(n^2)$ 时，Kruskal算法却比Prim算法好得多。

4.7 多机调度问题

多机调度问题要求给出一种作业调度方案,使所给的 n 个作业在尽可能短的时间内由 m 台机器加工处理完成。

约定,每个作业均可在任何一台机器上加工处理,但未完工前不允许中断处理。作业不能拆分成更小的子作业。

这个问题是**NP完全问题**,到目前为止还没有有效的解法。对于这一类问题,用贪心选择策略有时可以设计出较好的近似算法。

4.7 多机调度问题

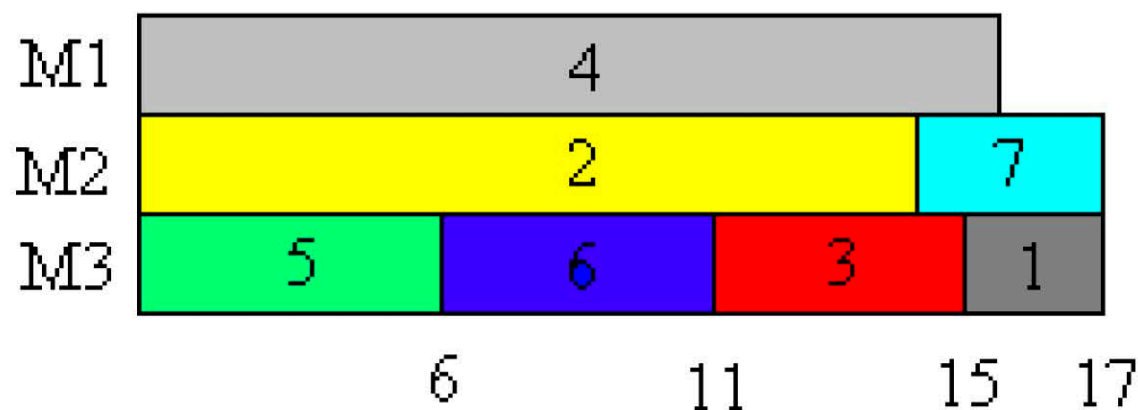
采用**最长处理时间作业优先**的贪心选择策略可以设计出解多机调度问题的较好的近似算法。

按此策略，当 $n \leq m$ 时，只要将机器 i 的 $[0, t_i]$ 时间区间分配给作业 i 即可。

当 $n > m$ 时，首先将 n 个作业依其所需的处理时间从大到小排序。然后依此顺序将作业分配给空闲的处理机。算法所需的计算时间为 $O(n \log n)$ 。

4.7 多机调度问题

例如，设7个独立作业{1,2,3,4,5,6,7}由3台机器M1、M2和M3加工处理。各作业所需的处理时间分别为{2,14,4,16,6,5,3}。按算法**greedy**产生的作业调度如图所示，所需的加工时间为17。



第四章 小结

重点：

1) 贪心选择性质

2) 最优子结构性质

难点：

贪心算法解题过程：

- 需证明每一步所做的贪心选择最终导致问题的整体最优解
- 每一步贪心选择后，原问题变为规模更小的问题
- 通过每一步贪心选择，最终可得到问题的整体最优解。

分治法、动态规划、贪心对比

类型	分治法	动态规划	贪心算法
子问题结构	每个子问题不同	很多子问题重复	一个子问题
子问题数	多个子问题	多个子问题	一个子问题
子问题独立	独立	子问题相关	独立
贪心选择性质	无	无	满足
最优子结构	无要求	必须满足	必须满足
问题与求解次序	自顶向下求解子问题 (递归)	自底向上解子问题，再 求更大规模问题	先贪心选择，再解决子 问题